

FINAL REPORT

PROBLEM STATEMENT AND DATASET:

VIRTUAL WORLD TRANSACTIONS ARE SUBJECTED TO ILLICIT FRAUD transactions thereby reducing the efficiency of metaverse system subjecting it to money laundering and high risk transactions so there is need for robust system to mitigate these anomaly

DATA DICTIONARY:

- Timestamp: Date and time of the transaction.
- Hour of Day: Hour part of the transaction timestamp.
- Sending Address: Blockchain address of the sender.
- Receiving Address: Blockchain address of the receiver.
- Amount: Transaction amount in a simulated currency.
- Transaction Type: Categorization of the transaction (e.g., transfer, sale, purchase, scam, phishing).
- Location Region: Simulated geographical region of the transaction.
- IP Prefix: Simulated IP address prefix for the transaction.
- Login Frequency: Frequency of login sessions by the user, varying by age group.
- Session Duration: Duration of activity sessions in minutes.
- Purchase Pattern: Behavioral pattern of purchases (e.g., focused, random, high-value).
- Age Group: Categorization of users into new, established, and veteran based on their activity history.
- Risk Score: Calculated risk score based on transaction characteristics and user behavior.
- Anomaly: Risk level assessment (e.g., high_risk, moderate_risk, low_risk)

OVERVIEW OF PROCEDURE:

Steps taken after obtaining the data from kaggle:

- We had studied the data in detail analysed the dataset
- We had done preprocessing of data removed the null and missing values
- Treated our outliers
- From the dataset we had used 3 target variable mainly: anomaly, risk score and transaction type
- We had prepared a pair plot to look into correlation amongst various variables
- We had done univariate, bivariate, multivariate analysis of all independent variables
- We had carried forward visualisation
- We had tried PCA and unsupervised learning techniques
- We had imputed missing value and carried forward with linear regression
- Seeing its poor performance carried forward with ridge and lasso
- Despite the hyperparameter tuning the model accuracy was not so good
- So used categoric target transaction type and carried forward decision

tree

- Even this model accuracy score was poor so carried out the decision tree with anomaly as target
- To improve this model tried random forest but there was no significant improvement
- There were no significant improvement
- Tried gradient boosting and it showed 82% accuracy

MODEL EVALUATION:

Class 0 (Low Risk):

- Precision: 0.300.300.30 – When the model predicts "Low Risk," it is correct 30% of the time.
- Recall: 0.010.010.01 – The model identifies only 1% of actual "Low Risk" cases.
- F1-Score: 0.020.020.02 – The balance between precision and recall is poor, indicating the model struggles with this class.
- Support: 125112511251 – There are 1251 true "Low Risk" cases in the test data.

Class 1 (Moderate Risk):

- Precision: 0.820.820.82 – When the model predicts "Moderate Risk," it is correct 82% of the time.
- Recall: 0.990.990.99 – The model identifies 99% of actual "Moderate Risk" cases.
- F1-Score: 0.900.900.90 – The model performs very well for this class.
- Support: 128481284812848 – There are 12848 true "Moderate Risk" cases in the test data.

Class 2 (High Risk):

- Precision: 0.260.260.26 – When the model predicts "High Risk," it is correct 26% of the time.
- Recall: 0.030.030.03 – The model identifies only 3% of actual "High Risk" cases.
- F1-Score: 0.050.050.05 – The balance between precision and recall is poor for this class.
- Support: 162116211621 – There are 1621 true "High Risk" cases in the test data.

Overall Metrics:

Accuracy:

- 0.810.810.81 – The model correctly predicts the risk level for 81% of all test cases.

Macro Average:

- Precision: 0.460.460.46, Recall: 0.340.340.34, F1-Score: 0.320.320.32
- These averages give equal weight to all classes, highlighting the model's poor performance for underrepresented classes (0 and 2).

Weighted Average:

- Precision: 0.720.720.72, Recall: 0.810.810.81, F1-Score: 0.740.740.74

- These averages are weighted by the support of each class, emphasizing the dominance of "Moderate Risk" (Class 1).
-

Interpretation:

- The model performs very well on Class 1 (Moderate Risk), which dominates the dataset in terms of support. This high performance boosts the overall accuracy.
 - The model struggles significantly with Class 0 (Low Risk) and Class 2 (High Risk). These classes are likely underrepresented in the dataset or have overlapping features with other classes, making them harder to distinguish.
 - The imbalance in support (most samples are Class 1) skews the model towards predicting Class 1 more often, leading to lower performance on the minority classes.
-

Next Steps for Improvement:

1.

Address Class Imbalance:

2.

- o Use techniques like oversampling (SMOTE) or undersampling to balance the dataset.
- o Adjust the model's class weights to penalize misclassification of minority classes more heavily.

3.

Feature Engineering:

4.

- o Investigate whether the features (e.g., amount, ip_prefix, etc.) have sufficient discriminatory power for all classes.
- o Explore additional features or transformations to improve separation between classes.

5.

Model Optimization:

6.

- o Experiment with more advanced models like Gradient Boosting, Random Forest, or Ensemble methods.
- o Use techniques like hyperparameter tuning to optimize the model's performance.

7.

Evaluation:

8.

- o Focus on metrics like the F1-Score or Precision-Recall curve, especially for minority classes, to better evaluate improvements.

Class-wise Performance:

High Risk (Class: high_risk):

- Precision: 0.000.000.00: None of the predictions for "high_risk" were correct.
- Recall: 0.000.000.00: The model failed to identify any actual "high_risk" instances.

- F1-Score: 0.000.000.00: Indicates complete failure in handling this class.
- Support: 125112511251: There are 1251 true "high_risk" instances in the test data.

Low Risk (Class: low_risk):

- Precision: 0.820.820.82: When the model predicts "low_risk," it is correct 82% of the time.
- Recall: 1.001.001.00: The model identified all "low_risk" cases.
- F1-Score: 0.900.900.90: Strong performance, reflecting both high precision and recall.
- Support: 128481284812848: The largest class in the dataset.

Moderate Risk (Class: moderate_risk):

- Precision: 0.500.500.50: Half of the "moderate_risk" predictions are correct.
 - Recall: 0.000.000.00: The model fails to identify any true "moderate_risk" instances.
 - F1-Score: 0.000.000.00: Reflects poor performance.
 - Support: 162116211621: Moderate class size but neglected by the model.
-

Overall Metrics:

Accuracy:

- 0.820.820.82: The model correctly classifies 82% of all test samples. However, this is heavily influenced by the dominant "low_risk" class.

Macro Average:

- Precision: 0.440.440.44, Recall: 0.330.330.33, F1-Score: 0.300.300.30
 - o Treats all classes equally, highlighting poor performance for "high_risk" and "moderate_risk."

Weighted Average:

- Precision: 0.720.720.72, Recall: 0.820.820.82, F1-Score: 0.740.740.74
 - o Weighted by the size of each class, emphasizing the model's bias towards "low_risk."
-

Key Observations:

1.

Class Imbalance:

2.

- o The "low_risk" class dominates the dataset, leading the model to prioritize it at the expense of minority classes ("high_risk" and "moderate_risk").

3.

Missed Classes:

4.

- o The model entirely neglects "high_risk" and "moderate_risk," resulting in recall and F1-scores of 0.000.000.00 for these classes.

5.

Overfitting to Majority Class:

6.
 - o The perfect recall for "low_risk" (1.001.001.00) suggests the model is heavily biased towards this class.
-

Suggested Improvements:

1.
Address Class Imbalance:
 - 2.
 - o Use techniques such as SMOTE (Synthetic Minority Over-sampling Technique), ADASYN, or undersampling of the majority class to balance the dataset.
 - o Adjust the model's class weights (class_weight='balanced' or custom weights).
3.
Feature Engineering:
 - 4.
 - o Investigate whether the features adequately represent the minority classes.
 - o Create new features or transformations that better capture "high_risk" and "moderate_risk" behaviors.
5.
Hyperparameter Tuning:
 - 6.
 - o Tune parameters like learning rate, maximum depth, and subsample ratio to improve generalization.
7.
Ensemble Techniques:
 - 8.
 - o Combine Gradient Boosting with other models (e.g., Random Forest or Logistic Regression) to improve minority class detection.
 - o Use stacking or voting classifiers for better performance across all classes.
9.
Evaluation Metrics:
 - 10.
 - o Focus on Precision-Recall metrics or F1-score for minority classes, as accuracy is misleading in imbalanced datasets.

DECISION TREE

ROC-AUC Scores:

- High Risk (0.64): The model distinguishes "high_risk" cases reasonably well, though there's room for improvement.
- Low Risk (0.59): A score slightly above random chance (0.500.500.50) suggests poor discrimination for this class.
- Moderate Risk (0.66): Slightly better than "low_risk," but still not ideal.

Interpretation: ROC-AUC values closer to 1.0 indicate better performance. The model struggles, especially with the "low_risk" class, despite its dominant presence in the dataset.

Classification Metrics:

High Risk:

- Precision (0.25): Only 25% of the predicted "high_risk" cases are correct.
- Recall (0.31): The model identifies 31% of actual "high_risk" instances.
- F1-Score (0.28): Low precision and recall result in a poor balance of both metrics.
- Support: 190619061906, indicating a smaller but significant presence in the dataset.

Low Risk:

- Precision (0.83): The model is highly accurate when predicting "low_risk."
- Recall (0.84): It identifies 84% of actual "low_risk" instances.
- F1-Score (0.84): Consistent precision and recall lead to strong overall performance.
- Support: 191981919819198, the majority class, benefiting from the model's bias.

Moderate Risk:

- Precision (0.29): About 29% of the predicted "moderate_risk" cases are correct.
 - Recall (0.21): The model identifies only 21% of actual "moderate_risk" instances.
 - F1-Score (0.24): Indicates poor performance due to low recall and precision.
 - Support: 247624762476, slightly more frequent than "high_risk."
-

Overall Metrics:

- Accuracy (0.73): About 73% of predictions are correct, but heavily influenced by the "low_risk" majority.
 - Macro Average:
 - o Precision (0.46), Recall (0.45), F1-Score (0.45): Averaged equally across all classes, exposing struggles with minority classes.
 - Weighted Average:
 - o Precision (0.73), Recall (0.73), F1-Score (0.73): Weighted by class size, reflecting strong performance on "low_risk."
-

Key Observations:

1. Imbalance Handling:
 - o The model performs well on the "low_risk" class but struggles with "high_risk" and "moderate_risk." This is a typical symptom of class imbalance.
 2. Model Bias:
 - o The model focuses on the majority class, neglecting minority classes despite their significance.
-

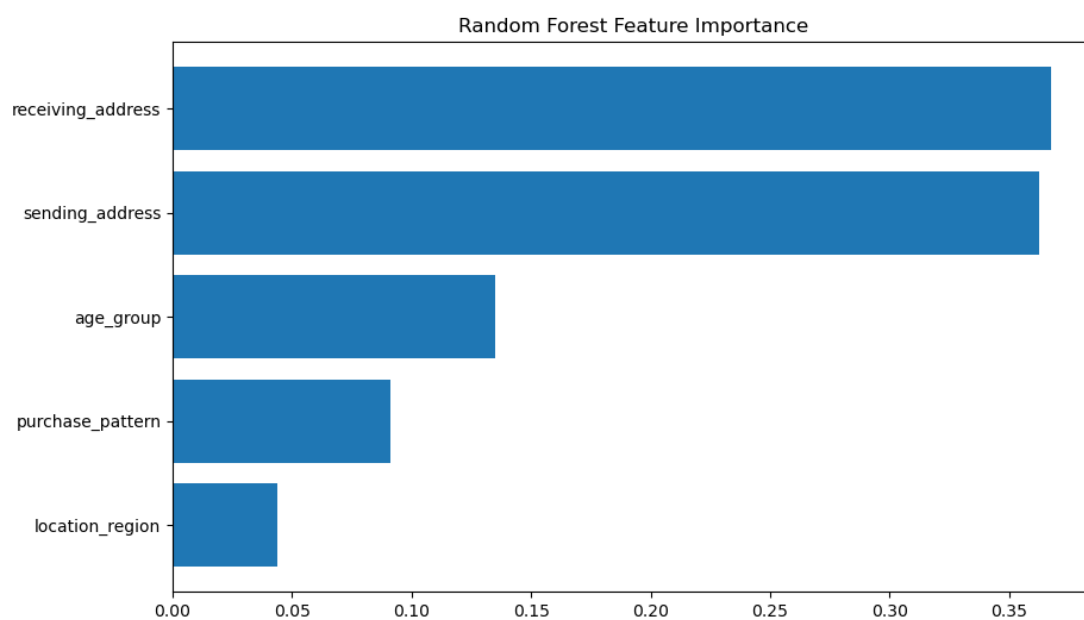
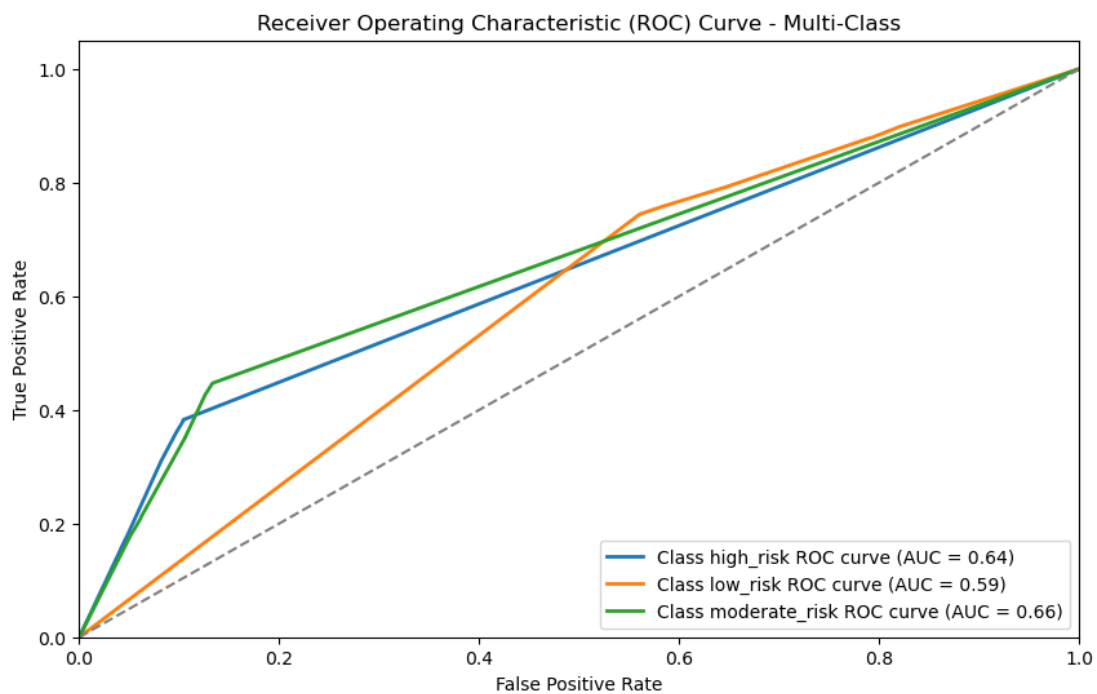
Next Steps for Improvement:

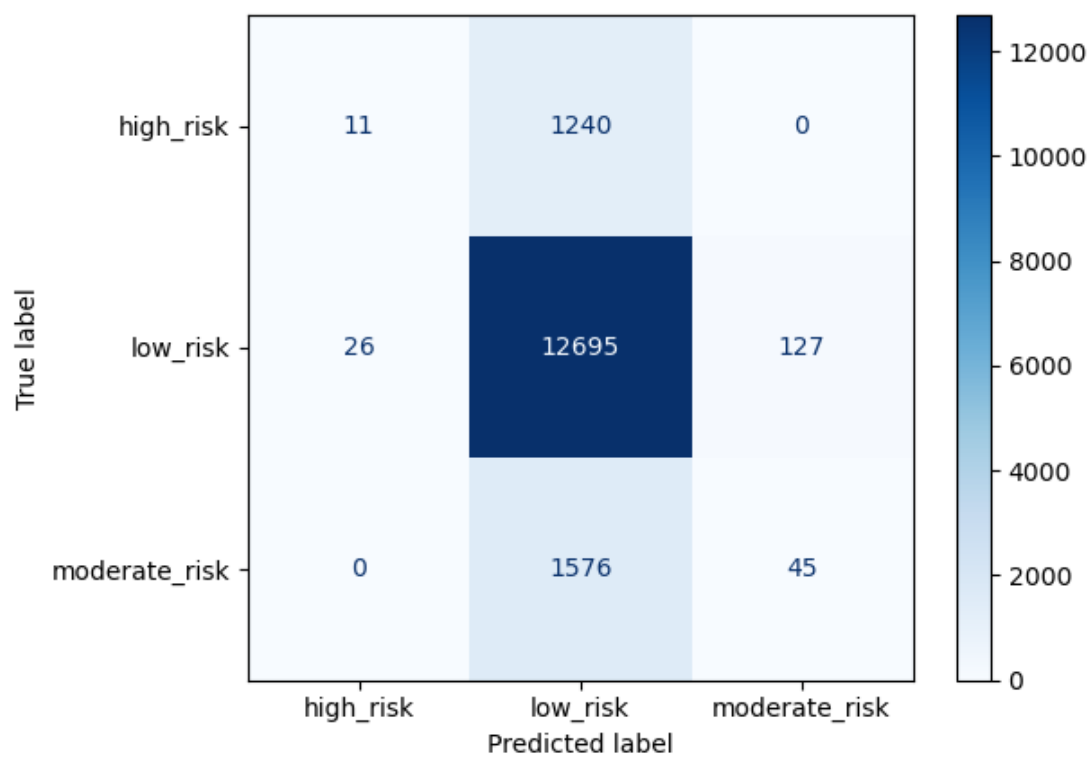
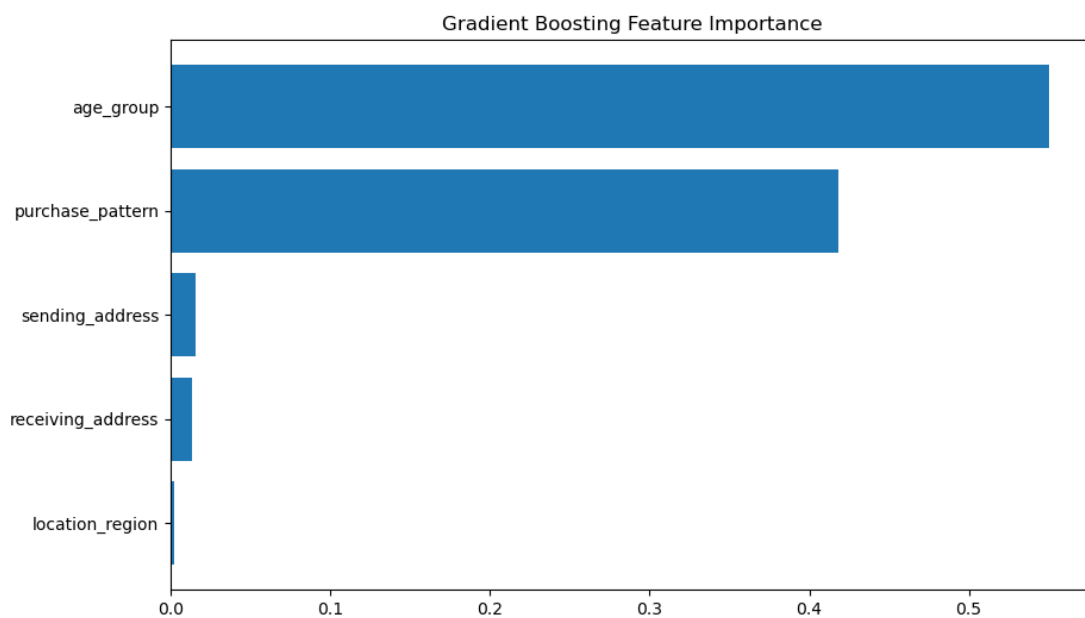
1. Address Class Imbalance:

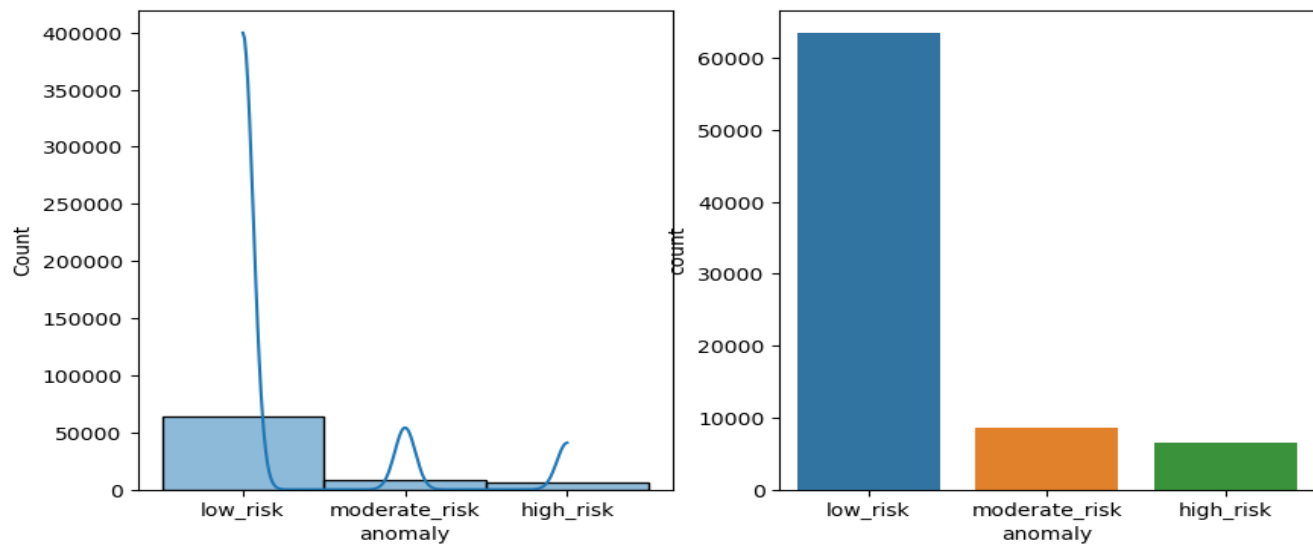
- o Revisit sampling strategies (e.g., SMOTE, undersampling, or oversampling).
- o Assign custom class weights to penalize incorrect predictions for minority classes.

VISUALISATIONS:









Limitation:

Major problem encountered is the calculation of high risk which is failing with respect to our dataset

REFLECTION:

Would look into more evenly spread data
Collect data directly from resources